Практическое задание №8

Создать новый блокнот и изменить название в соответствии с примером изменив соответствующие данные (Пример: Практическое задание №1. Иванов И.И. Группа 12345.). Блокнот должен содержать заголовок, краткое описание задач и код, выполняющий задачи:

Задача 1. Создайте пустой список с именем **nums**. Предложите пользователю последовательно вводить числа. После ввода каждого числа добавьте его в конец списка **nums** и выведите список. После того как пользователь введет три числа, спросите, хочет ли он оставить последнее введенное число в списке. Если пользователь ответит «нет», удалите последний элемент из списка. Выведите список.

Задача 2. Напишите программу, которая будет запрашивать у пользователя направление сортировки и целочисленные значения, которые сохранит в виде списка. Индикатором окончания ввода значений должен служить ноль. Затем программа должна вывести на экран все введенные пользователем числа (кроме нуля) в порядке возрастания или убывания – по одному значению в строке. Используйте для сортировки либо метод `sort`, либо функцию `sorted`.

Задача 3. В данном упражнении вам предстоит разработать программу, в которой у пользователя будет запрошен список слов, пока он не оставит строку ввода пустой. После этого на экране должны быть показаны слова, введенные пользователем, но без повторов, – каждое по одному разу. При этом слова должны быть отображены в том же порядке, в каком их вводили с клавиатуры. Например, если пользователь на запрос программы введет следующий список слов:

```
first
second
first
third
second
```

программа должна вывести:

```
first
second
third
```

Задача 4. При анализе собранных по результатам научных экспериментов данных зачастую возникает необходимость избавиться от экстремальных значений, прежде чем продолжать двигаться дальше. Напишите функцию, создающую копию списка с исключенными из него **n** наибольшими и наименьшими значениями и возвращающую ее в качестве результата. Порядок следования элементов в измененном списке не обязательно должен в точности совпадать с источником.

В основной программе должна быть продемонстрирована работа вашей функции. Для начала попросите пользователя ввести целые числа, затем соберите их в список и вызовите написанную вами ранее функцию. Выведите на экран измененную версию списка вместе с оригинальной. Если пользователь введет менее четырех чисел, должно быть отображено соответствующее сообщение об ошибке

Задача 5. Подмножеством элементов, или подсписком называется список, являющийся составной частью большего списка. Напишите функцию, возвращающую список, содержащий все возможные подсписки заданного. Например, в число подсписков списка [1, 2, 3] входят следующие: [ ], [1], [2], [3], [1, 2], [2, 3] и [1, 2, 3]. Заметьте, что ваша функция должна вернуть как минимум один пустой список, гарантированно являющийся подсписком для любого списка. Напишите основную программу, демонстрирующую работу функции применительно к нескольким исходным спискам

Задача 6. Стандартная игральная колода состоит из 52 карт. Каждая карта соответствует одной из четырех мастей (пики, червы, бубны и трефы) и одному из 13 номиналов (от 2 до 10, валет (J), дама (Q), король (K) и туз (A)).

Таким образом, каждая игральная карта может быть представлена при помощи двух символов. Первый из них указывает на номинал карты (от 2 до 9, T (десятка), J, Q, K или A), а второй – на масть (s = пики (spades), h = червы (hearts), d = бубны (diamonds) и c = трефы (clubs)). В таблице представлены некоторые из возможных обозначений игральных карт.

Карта	**Обозначение**
Валет пик	Js
Двойка треф	2c
Десятка бубен	Td
Туз червей	Ah
Девятка пик	9s

Начните с написания функции **createDeck**. В ней должны использоваться циклы для создания полной колоды карт путем сохранения в список двухсимвольных аббревиатур всех 52 карт. Именно этот список и будет возвращаемым из данной функции значением. На вход функция **createDeck** принимать параметры не будет.

Напишите вторую функцию с именем **shuffle**, которая будет случайным образом перетасовывать карты в списке. Одна из техник тасования колоды заключается

в проходе по элементам и перестановке их с любым другим случайным элементом в этом списке. Вы должны создать свой собственный цикл для тасования карт в колоде, а не пользоваться стандартной функцией **shuffle** языка Python.

Используйте обе созданные функции в основной программе, в которой должна отображаться колода карт до и после тасования.

Задача 7. Во многих карточных играх после процедуры тасования колоды каждый игрок получает на руки определенное количество карт. Напишите функцию **deal**, принимающую на вход три параметра: количество игроков, количество раздаваемых каждому из них карт и саму колоду. Функция должна возвращать список рук, которые были розданы игрокам. При этом каждая рука, в свою очередь, тоже является списком из входящих в нее карт.

Во время раздачи карт игрокам функция параллельно должна удалять розданные карты из переданной ей третьим параметром колоды. Также принято раздавать карты каждому игроку по одной строго по очереди. Придерживайтесь этих принципов и при написании своей функции.

Воспользуйтесь своими наработками из задачи 6 при построении структуры основной программы. Вам необходимо создать колоду карт, перетасовать ее и раздать четырем игрокам по пять карт. Выведите на экран карманные карты всех игроков, находящихся в раздаче, а также оставшиеся в колоде карты.